10.03.2026

История и архитектура Unix. Системные вызовы и ядра. Поиск подстрок (продолжение).

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

Новосибирский государственный университет

*НАСТОЯЩАЯ НАУКА

Давайте познакомимся

Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.

Новосибирский государственный университет
*НАСТОЯЩАЯ НАУКА

Адженда

История и архитектура Unix

30 минут

Системные вызовы и ядра

30 минут

Поиск подстрок (продолжение)

30 минут

Адженда

История и архитектура Unix

30 минут

Системные вызовы и ядра

30 минут

Поиск подстрок (продолжение)

30 минут

Введение

Unix — первая операционная система, реализованная на достаточно высокоуровневом языке программирования, C, который, в свою очередь, был разработан специально для этой цели и получил известность и влияние благодаря Unix. Хотя, конечно, стоит отметить, что C больше не считается языком программирования высокого уровня.

Если бы в 1970-е и 1980-е годы инженеры в Bell Labs решили перейти с C на другой язык для разработки новой версии Unix, то мы бы сегодня говорили именно об этом.

Высказывание Денниса М. Ритчи, одного из пионеров C, о роли Unix в успехе данного языка:

«Успех самой системы Unix, несомненно, сыграл самую важную роль; благодаря ему этот язык стал доступен сотням тысяч людей. С другой стороны, конечно, использование C в Unix обеспечило переносимость этой системы на широкий спектр компьютеров, что было важно для ее успеха.»

История Unix

Multics OS и Unix

Еще до Unix существовала система ***Multics OS*** — совместный проект MIT, General Electric и Bell Labs, запущенный в 1964 году. Это был первый в мире пример рабочей и безопасной операционной системы, что обеспечило ее успех. Multics устанавливали везде, от университетов до правительственных учреждений. Если перенестись обратно в наше время, то все современные ОС заимствуют те или иные идеи из Multics.

В 1969 году по разным причинам, часть сотрудников Bell Labs, особенно такие пионеры, как Кен Томпсон и Деннис Ритчи, разочаровались в Multics, в результате чего проект был заброшен. Компания Bell Labs разработала собственную, более простую и эффективную операционную систему - Unix.

Вдобавок стоит отметить, что компания Bell Labs работала над новой распределенной операционной системой под названием Plan 9, которая была основана на проекте Unix.

Полагаю, достаточно сказать, что система Unix являлась упрощенным выражением идей и инноваций, представленных в Multics; она не была чем-то новым.

Сходства и различия Unix и Multics.

- Внутренняя структура обеих ОС имеет многослойную архитектуру. Это значит, их архитектура состоит примерно из одних и тех же колец. Особенно это касается ядра и командной оболочки. Кроме того, Unix и Multics предоставляют ряд собственных утилит, таких как ls и pwd.
- Системе Multics для работы требовались дорогие ресурсы и оборудование. Это был один из основных недостатков, который открыл дорогу.
- Архитектура Multics была сложной. В этом состояла основная причина разочарования работников Bell Labs. Систему Unix изначально пытались сделать простой. В первой версии не было даже многозадачности и многопользовательского режима!

BCPL и B

Язык программирования BCPL был создан Мартином Ричардсом для написания компиляторов. Сотрудники Bell Labs познакомились с этим языком во время работы над Multics. После закрытия проекта компания Bell Labs начала писать Unix на ассемблере. В те времена это был единственный приемлемый вариант!

Факт, что участники проекта Multics использовали в разработке PL/1, считался чем-то необычным, но являлся доказательством того, что операционную систему можно успешно написать на языке программирования высокого уровня. И это стало основной причиной, почему разработка Unix была переведена на другой язык.

Кен Томпсон и Деннис Ритчи продолжили попытки написания модулей операционной системы на языке, отличном от ассемблера. Они попробовали использовать BCPL, но оказалось, что для применения этого языка на компактных компьютерах наподобие DEC PDP-7 его необходимо модифицировать. Эти модификации привели к появлению языка программирования B.

Язык B, будучи системным языком программирования, имел недостатки. В нем не было системы типов, из-за чего в каждой операции он позволял работать только с машинными словами (а не с байтами). Это затрудняло его применение на компьютерах с разной длиной машинных слов.

Язык B со временем претерпел ряд изменений, которые привели к появлению языка NB (new B — «новый B»), унаследовавшего структуры из B. В языке B эти структуры не имели типа, но в C стали типизированными. Наконец, в 1973 году вышла четвертая версия Unix, написанная с использованием C + assembler.

Путь к С

Недостатки В, которые привели к появлению С.

- Язык В позволял работать только с машинными словами в памяти.
- Язык В не поддерживал типы. Если более точно, то В был однотипным языком.
- Отсутствие типов привело к тому, что многие операции, ориентированные на байты (такие как алгоритмы для работы со строками), были неэффективно написаны на В. Причина — язык В использовал для работы с памятью слова, а не байты, что не позволяло эффективно управлять многобайтными типами данных, такими как целые числа и строки.
- Язык В не поддерживал операции с плавающей запятой. В то время подобные операции становились все более доступными в новом оборудовании, но их поддержка в языке В отсутствовала.
- Несмотря на существование таких компьютеров, как PDP-1, которые могли работать с памятью побайтно, язык В демонстрировал низкую эффективность при адресации байтов памяти.

Недостатки языка В вынудили Денниса Ритчи создать новый язык, NB (new B), который в итоге превратился в С.

Этот новый язык пытался исправить изъяны и трудности, присущие В, и стал фактически стандартом в мире системного программирования, заменив ассемблер. Менее чем через десять лет новые версии Unix уже были полностью написаны на С, и с тех пор все ОС, основанные на Unix, полагаются на данный язык и его важнейшую роль в системе.

На сегодня у него нет конкурентов!

В отличие от многих других языков программирования С является стандартом ISO.

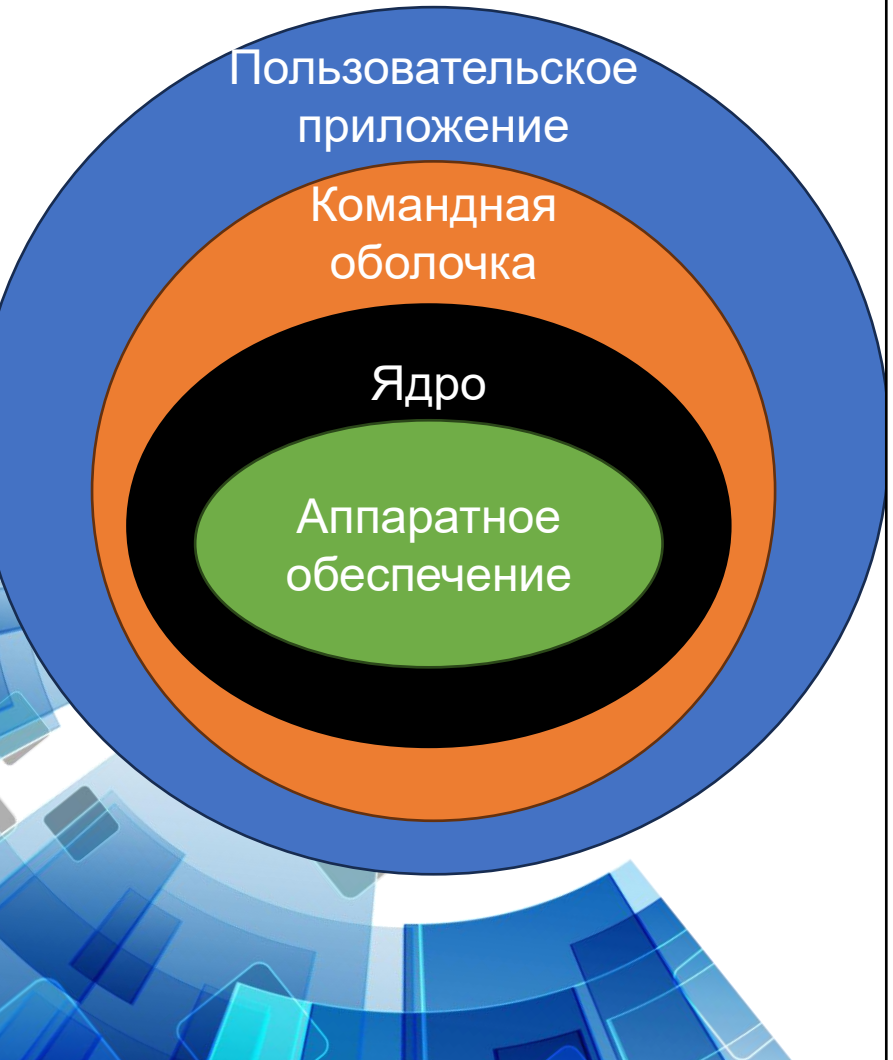
Архитектура Unix

Философия

- Система Unix проектировалась и разрабатывалась в основном для программистов, а не для обычных пользователей. В связи с этим многие требования, относящиеся к пользовательскому интерфейсу и взаимодействию с пользователями, не являются частью архитектуры Unix.
- Unix состоит из множества мелких и простых программ. Каждая из них предназначена для выполнения небольшой и простой задачи. Существует множество примеров таких программ, включая ls, mkdir, ifconfig, grep и sed.
- Unix поощряет программистов разбивать свои большие и сложные проекты на мелкие и простые программы.
- Каждая мелкая и простая программа должна уметь подавать свой вывод на вход другой программе, продолжая цепочку выполнения. Таким образом, мелкие программы можно объединять в цепочку для выполнения сложных задач. Например, ls -l | grep a.out.
- Система Unix строго ориентирована на работу с текстом. Вся конфигурация хранится в текстовых файлах, и командная строка тоже текстовая. Скрипты командной оболочки тоже представляют собой текстовые файлы с простым синтаксисом для написания алгоритмов.
- Unix поощряет выбор простоты перед совершенством.
- Программы, написанные для определенной операционной системы, совместимой с Unix, должны легко переноситься на другие системы данного семейства. Это в основном достигается благодаря наличию единой кодовой базы.

Какую роль в этом играет язык C? Дело в том, что почти все ключевые элементы (то есть мелкие и простые программы, лежащие в основе Unix), о которых шла речь выше, написаны на данном языке. Пример: исходный код программы ls из NetBSD.

Многослойная структура Unix



Основная задача операционной системы — дать пользователю возможность взаимодействовать с оборудованием.

Одна из важнейших задач, стоящих перед Unix: сделать оборудование доступным для программ, которые хотят к нему обращаться.

Ядро - это самая важная часть операционной системы. Оно ближе всего находится к оборудованию и служит оберткой для предоставления возможностей, которыми обладает данное оборудование.

Командная оболочка – это обертка, которая позволяет пользовательским приложениям взаимодействовать с ядром и применять его многочисленные функции. Оно также содержит набор библиотек, полностью написанных на C, с помощью которых программист может разрабатывать новые приложения для Unix.

Используя библиотеки, описанные в спецификации SUS (Simple Unix Specification — простая спецификация Unix), кольцо командной оболочки должно предоставлять программистам стандартный и строго определенный интерфейс.

Пользовательские приложения – кольцо, состоящее из прикладных программ, предназначенных для выполнения в системах Unix. Речь идет о базах данных, веб-сервисах, почтовых серверах, браузерах, электронных таблицах и текстовых процессорах. Эти приложения должны использовать API и инструменты, предоставляемые командной оболочкой, не обращаясь к ядру напрямую (через системные вызовы, которые мы вскоре обсудим).

Характерная черта архитектуры Unix — тот факт, что внешние кольца должны взаимодействовать с внутренними через определенный интерфейс.

Интерфейс командной оболочки для ПП

Чтобы применить возможности, доступные в системе Unix, живой пользователь работает либо с терминалом, либо с отдельной графической программой, такой как браузер. И то и другое принадлежит к категории пользовательских приложений. Оперативная память, центральный процессор, сетевой адаптер и жесткие диски — все это типичные примеры аппаратного обеспечения, которое большинство Unix-программ задействуют через API кольца командной оболочки.

С точки зрения разработчика, приложение мало чем отличается от программы. Но в понимании пользователя приложение — программа, с которой можно взаимодействовать через графический или консольный интерфейс (GUI и CLI соответственно), а программа — просто программный компонент, который выполняется на компьютере без какого-либо пользовательского интерфейса (как в случае с сервисом).

Как написать такую программу на C, которую можно собрать в разных версиях Unix и на разных аппаратных платформах?

Ответ прост: все системы Unix предоставляют для своей командной оболочки один и тот же интерфейс прикладного программирования (Application Programming Interface, API). Фрагмент кода на языке C, который использует только этот стандартный интерфейс, можно собрать и запустить на любой Unix-системе.

API — куча заголовочных файлов с объявлениями.

На самом деле заголовок stdio.h не является частью C, несмотря на то что он и объявленные в нем функции фигурируют во всех книгах о данном языке. Это часть стандартной библиотеки C, описанной в стандарте SUS. Программе, написанной на C для Unix, ничего не известно о реализации тех или иных функций, таких как printf или fopen. Иными словами, программы во внешнем кольце воспринимают командную оболочку как некий черный ящик.

Интерфейс командной оболочки для ПП

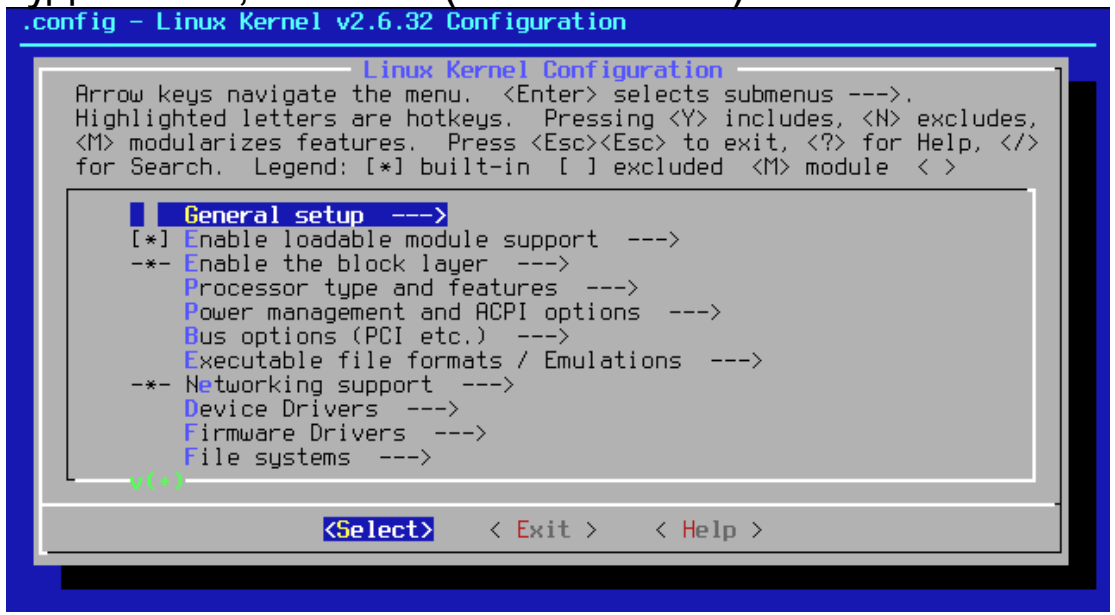
В стандарте SUS собраны различные API, предоставляемые кольцом командной оболочки. Этот стандарт развивается консорциумом The Open Group и имеет несколько версий, вышедших с момента создания Unix.

- ***Системные интерфейсы*** — сюда входят все функции, которыми могут пользоваться любые программы на языке C. В стандарте SUS v4 предусмотрена 1191 функция, которая должна быть реализована в Unix-системе.
- **Заголовочные интерфейсы** — список заголовочных файлов, которые могут быть доступны в Unix-системе, совместимой с SUS v4. В этой версии SUS перечислены 82 заголовочных файла, к которым может обращаться любая программа на языке C. Если пройтись по этому списку, то можно найти много известных заголовков, таких как stdio.h, stdlib.h, math.h и string.h.
- ***Вспомогательные интерфейсы*** — список консольных утилит или программ командной строки, которые должны быть доступны в Unix-системе, совместимой с SUS v4. Если пройтись по таблицам, то можно встретить много знакомых нам команд, таких как mkdir, ls, cp, df, bc; всего их насчитывается 160.

- ***Сценарный интерфейс*** — язык, который используется для написания скриптов командной оболочки. С его помощью в основном автоматизируют задачи, в которых применяются консольные утилиты. Этот интерфейс обычно называют языком командной оболочки (или командной строки).
- ***Интерфейсы XCURSES*** — набор интерфейсов, которые позволяют программе, написанной на C, взаимодействовать с пользователем с помощью минималистичного текстового GUI.

Интерфейс командной оболочки для ПП

В SUS v4 интерфейсу XCURSES отводится 379 функций, размещенных в трех заголовках, а также четыре утилиты.
Многие программы до сих пор применяют XCURSES для более удобного взаимодействия с пользователем. Следует отметить, что интерфейсы на основе XCURSES не нуждаются в графической подсистеме. Благодаря этому с ними можно работать удаленно, по SSH (Secure Shell).



Конфигурационное меню на основе ncurses

Интерфейс командной оболочки для ПП

SUS не описывается иерархия файловой системы и то, где можно найти те или иные заголовочные файлы. Этот стандарт лишь указывает на то, какие заголовки должны присутствовать и быть доступны в системе. Согласно широко распространенному соглашению о заголовочных файлах они должны находиться либо в /usr/include, либо /usr/local/include, но окончательное решение по-прежнему остается за операционной системой и пользователем.

libc — это набор функций, размещенных в определенных заголовочных файлах в соответствии с SUS, плюс статические и динамические библиотеки с реализациями доступных функций.

Определение libc тесно связано с процессом стандартизации систем Unix. Любая программа на языке C, разработанная в системе Unix, использует libc для взаимодействия с более низкими уровнями ядра и аппаратного обеспечения.

Не все операционные системы полностью совместимы с Unix. Например, Microsoft Windows и ОС с ядром Linux (Android).

Unix-совместимая система полностью соответствует стандартам SUS, чего нельзя сказать о Unix-подобной системе, которая имеет лишь частичную совместимость.

Появление множества Unix-подобных операционных систем, особенно после рождения Linux, создало необходимость в классификации этого конкретного подмножества SUS. Данный стандарт был назван POSIX (Portable Operating System Interface — переносимый интерфейс операционных систем). Можно сказать, что POSIX — подмножество SUS, с которым совместимы Unix-подобные системы.

Unix-подобные ОС, включая большинство дистрибутивов Linux, изначально POSIX-совместимы. Вот почему с Ubuntu можно работать так же, как с FreeBSD.

Интерфейс ядра для кольца КО

Нам нужен был процесс компиляции и механизм компоновки, чтобы спроектировать операционную систему, которая предоставляет интерфейс и реализует набор библиотечных файлов.

Заметьте, что каждая возможность языка C играет на руку Unix. Чем лучше вы будете понимать отношения между C и Unix, тем более очевидной будет для вас их тесная связь.

Для работы с ядром libc (или функции в кольце командной оболочки) в основном используют системные вызовы. Чтобы объяснить этот механизм и показать, в каком месте многослойной модели применяются системные вызовы, нам нужен реальный пример.

Нам также следует выбрать конкретную реализацию libc, чтобы иметь возможность проанализировать исходники и найти соответствующие системные вызовы. Например, FreeBSD. Это Unix-подобная операционная система, которая является ответвлением от BSD Unix. Исходники libc этой системы можно найти в директории lib/libc.

подключается функция sleep из кольца ядра

```
#include <unistd.h>
int main(int argc, char **argv)
{
    sleep(1);
    return 0;
}
```

Код подключает заголовочный файл unistd.h и вызывает функцию sleep; и то и другое — часть интерфейсов, доступных в SUS. Но что происходит дальше, особенно в функции sleep?

Системные вызовы инициируются кодом, написанным в реализации libc. На самом деле именно так вызываются процедуры ядра. В SUS и впоследствии в POSIX-совместимых системах была предусмотрена программа, которая позволяла отслеживать системные вызовы во время выполнения кода.

Интерфейс ядра для кольца КО

```
$ gcc main.c -o a.out
$ truss ./a.out //для FreeBSD
$ strace ./a.out //аналог, если нет в вашей ОС truss
mmap(0x0,32768,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANON,-1,0x0) = 34366160896
(0x800620000)
issetugid() = 0 (0x0)
...
openat(AT_FDCWD,"/lib/libc.so.7",O_RDONLY|O_CLOEXEC|O_VERIFY,00) = 3 (0x3)
...
nanosleep({ 1.000000000 }) = 0 (0x0)
sigprocmask(SIG_BLOCK,{ SIGHUP|SIGINT|SIGQUIT|SIGKILL|SIGPIPE|SIGALRM|
SIGTERM|SIGURG|SIGSTOP|SIGTSTP|SIGCONT|SIGCHLD|SIGTTIN|SIGTTOU|SIGIO|
SIGXCPU|SIGXFSZ|SIGVTALRM|SIGPROF|SIGWINCH|SIGINFO|SIGUSR1|
SIGUSR2 },{ }) = 0 (0x0)
...
sigprocmask(SIG_SETMASK,{ },0x0) = 0 (0x0)
exit(0x0)
process exit, rval = 0
```

В этом простом примере сделано много системных вызовов. Некоторые из них относятся к загрузке разделяемых объектных библиотек, особенно на этапе инициализации процесса. Полужирным шрифтом, открывает разделяемый объектный файл libc.so.7, содержащий непосредственную реализацию libc для FreeBSD; программа делает системный вызов nanosleep.

Интерфейс ядра для кольца КО

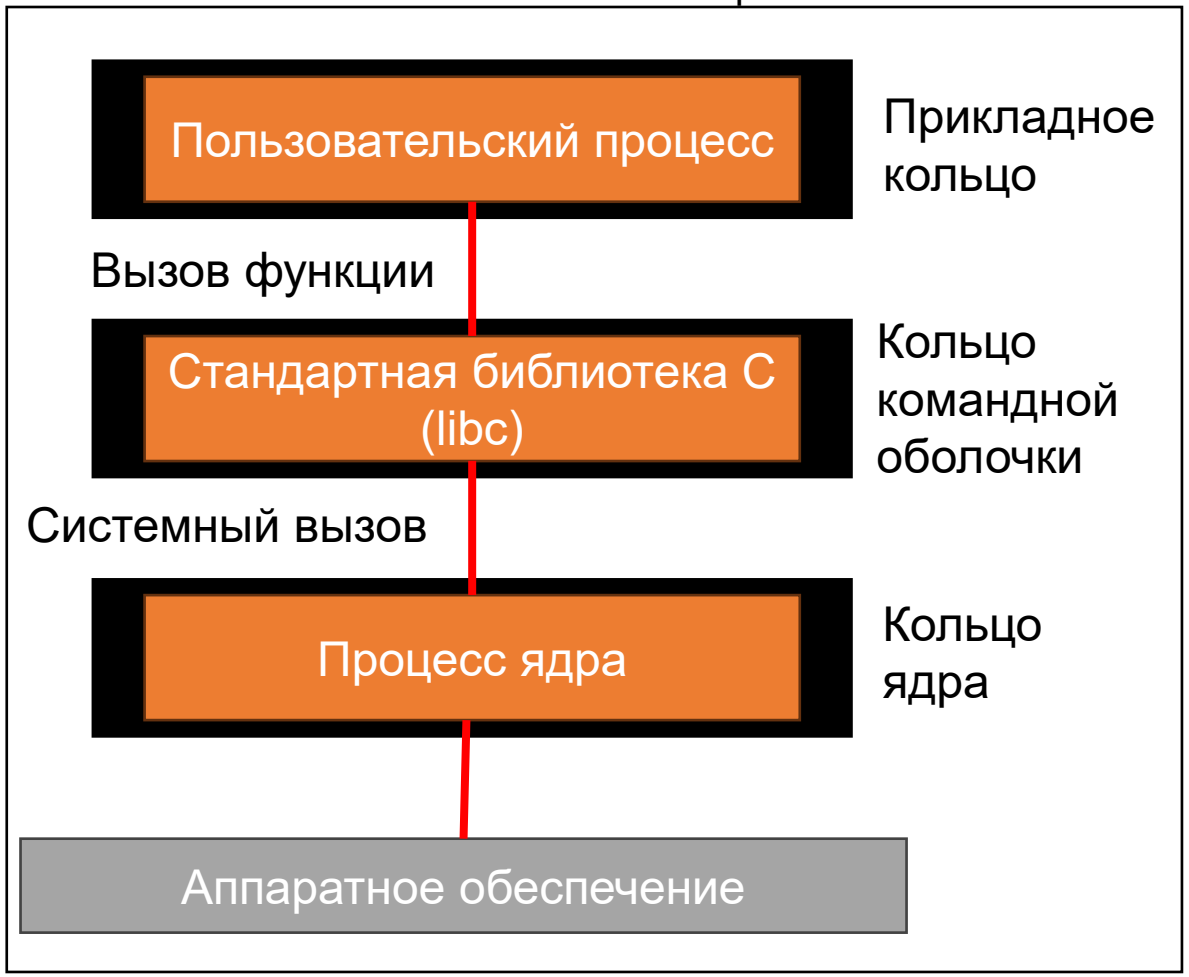
Теперь найдем то место в libc, где инициируется системный вызов.
Если зайти в каталог lib/libc и выполнить grep для sys_nanosleep, увидим следующие файлы:

```
$ git clone https://github.com/freebsd/freebsd-src.git
...
$ cd freebsd-src
$ git reset --hard bf78455d496
...
$ cd lib/libc
$ grep sys_nanosleep . -R
./include/libc_private.h:int            __sys_nanosleep(const struct timespec *,
struct timespec *);
./sys/interposing_table.c:      SLOT(nanosleep, __sys_nanosleep),
./sys/nanosleep.c:__weak_reference(__sys_nanosleep, __nanosleep);
./sys/Symbol.map:       __sys_nanosleep;
```

Как можно видеть в файле lib/libc/sys/interposing_table.c, функция nanosleep привязана к вызову __sys_nanosleep. Следовательно, при выполнении nanosleep всегда вызывается __sys_nanosleep.
В FreeBSD названия всех функций, которые являются системными вызовами, принято начинать с __sys.
В приведенном выше терминале есть еще один интересный участок. Файл lib/ libc/include/libc_private.h содержит объявления приватных и внутренних функций-оберток вокруг системных вызовов.

Ядро

Основное назначение кольца ядра состоит в управлении оборудованием, подключенным к системе, и предоставлении доступа к своим возможностям с помощью системных вызовов.



Ядро

- Процесс ядра — первое, что загружается и выполняется в системе. Только вслед за ним можно запускать пользовательские процессы.
- Процесс ядра существует в единственном экземпляре, при этом мы можем запустить сразу несколько пользовательских процессов.
- Процесс ядра создается в результате того, что загрузчик копирует его образ в основную память, а для создания пользовательского процесса применяются системные вызовы exec или fork, присутствующие в большинстве Unix-систем.
- Процесс ядра обрабатывает и выполняет системные вызовы, в то время как пользовательский их инициирует и ждет их выполнения.
- Процесс ядра имеет привилегированный доступ к физической памяти и всему подключенному оборудованию, в то время как пользовательский работает с виртуальной памятью, которая является отражением части физической памяти, и ничего не знает о ее физической структуре. Пользовательский процесс имеет управляемый и наблюдаемый доступ к ресурсам и оборудованию.

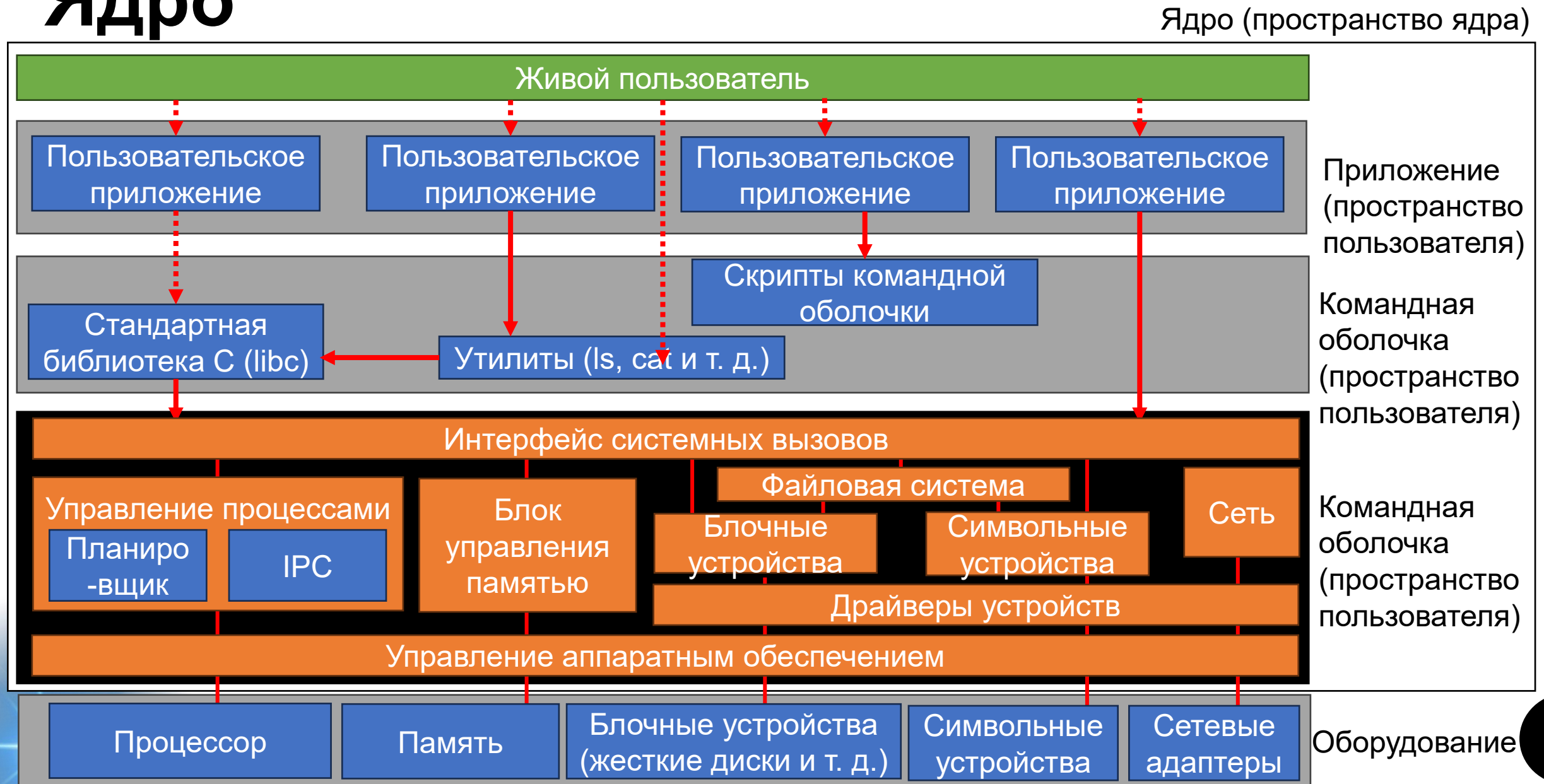
Среда выполнения операционной системы имеет два разных режима. Один предназначен для процесса ядра, а другой — для пользовательских процессов. Первый режим выполнения называется пространством ядра, а второй — пользовательским пространством..

Ядро

Внутреннюю структуру типичного Unix-ядра можно разделить по выполняемым задачам:

- ***Управление процессами.*** Ядро создает пользовательские процессы с помощью системного вызова. Среди прочих операций перед запуском нового процесса необходимо сначала выделить память и загрузить его инструкции.
- ***Межпроцессное взаимодействие*** (Inter-Process Communication, IPC). Пользовательские процессы, запущенные на одном компьютере, могут задействовать разные методы обмена данными, включая разделяемую память, каналы и доменные сокеты Unix.
- ***Планирование.*** Ядро управляет доступом к центральному процессору, пытаясь его балансировать. **Планирование** — процедура разделения процессорного времени между разными процессами в зависимости от их приоритета и значимости.
- ***Управление памятью.*** Ядро — единственный процесс, который видит всю физическую память и имеет к ней неограниченный доступ. Поэтому на него ложится разбиение ее на выделяемые страницы, назначение новых страниц процессам (при выделении кучи), освобождение памяти и многие другие сопутствующие процедуры.
- ***Начальная загрузка системы.*** После загрузки образа ядра в основную память его процесс должен запуститься и инициализировать пользовательское пространство. Обычно для этого создается первый пользовательский процесс с PID.
- ***Управление устройствами.*** Ядро должно иметь возможность управлять не только процессором и памятью, но и аппаратным обеспечением. Для этого предусмотрен слой абстракции. В Unix устройства привязываются к файлам, которые обычно хранятся в каталоге /dev.

Ядро


Ядро (пространство ядра)
Живой пользователь
Пользовательское приложение
Пользовательское приложение
Пользовательское приложение
Пользовательское приложение
Приложение (пространство пользователя)
Скрипты командной оболочки
Стандартная библиотека C (libc)
Утилиты (ls, cat и т. д.)
Командная оболочка (пространство пользователя)
Интерфейс системных вызовов
Управление процессами
Планиро -вщик
IPC
Блок управления памятью
Файловая система
Блочные устройства
Символьные устройства
Сеть
Драйверы устройств
Управление аппаратным обеспечением
Командная оболочка (пространство пользователя)
Процессор
Память
Блочные устройства (жесткие диски и т. д.)
Символьные устройства
Сетевые адаптеры
Оборудование

Аппаратное обеспечение

```
$ ls -l /dev
total 0
crw-r--r-- 1 root root     10, 235 Mar  2 15:24 autofs
drwxr-xr-x 2 root root         580 Mar  2 15:24 block
drwxr-xr-x 2 root root         100 Mar  2 15:24 bsg
crw-rw---- 1 root disk     10, 234 Mar  2 15:24 btrfs-control
drwxr-xr-x 3 root root          60 Mar  2 15:24 bus
drwxr-xr-x 2 root root        2740 Mar  2 15:24 char
crw--w---- 1 root tty       5,   1 Mar  2 15:24 console
...
lrwxrwxrwx 1 root root          28 Mar  2 15:24 log -> /run/systemd/journal/dev-log
crw-rw---- 1 root disk     10, 237 Mar  2 15:24 loop-control
...
lrwxrwxrwx 1 root root          15 Mar  2 15:24 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root          15 Mar  2 15:24 stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root root          15 Mar  2 15:24 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty       5,   0 Mar  2 15:27 tty
...
drwxr-xr-x 2 root root          60 Mar  2 15:24 vfio
crw-rw---- 1 root kvm      10, 238 Mar  2 15:24 vhost-net
drwxr-xr-x 2 root root          80 Mar  2 15:24 virtio-ports
```

Благодаря слою абстракции поверх аппаратного обеспечения система Unix поддерживает виртуальные устройства.

Адженда

История и архитектура Unix

30 минут

Системные вызовы и ядра

30 минут

Поиск подстрок (продолжение)

30 минут

Системные вызовы

Исследование системных вызовов

Правило: содержимое двух внутренних колец, ядра и аппаратного обеспечения не должно быть доступно напрямую из пространства пользователя. Иными словами, никакой пользовательский процесс не должен иметь прямого доступа к оборудованию, а также к внутренним структурам данных и алгоритмам ядра. Обращение к ним должно происходить через системные вызовы.

Чтобы прояснить ситуацию, можно рассмотреть пример того, как данные проходят от пространства пользователя к пространству ядра и обратно. Когда вам нужно прочитать файл с жесткого диска, вы пишете программу для кольца пользовательских приложений. Она задействует функцию ввода/вывода из libc под названием fread (или ее аналог) и в итоге выполняется в виде процесса в пространстве пользователя. Когда программа вызывает функцию fread, срабатывает ее реализация внутри libc.

Пока все это происходит в пользовательском процессе. В конечном счете реализация fread инициирует системный вызов, принимая три аргумента: дескриптор уже открытого файла, адрес буфера, выделенного в памяти процесса (который находится в пространстве пользователя) и длину данного буфера.

Когда реализация libc инициирует системный вызов, поток выполнения пользовательского процесса переходит к ядру, которое получает аргументы из пользовательского пространства и размещает их в пространстве ядра. Затем ядро читает файл, обращаясь к своему модулю файловой системы.

Исследование системных вызовов

После завершения операции read в кольце ядра прочитанные данные копируются в буфер в пространстве пользователя, указанный во втором аргументе функции fread; дальше поток выполнения переходит от системного вызова к пользовательскому процессу. Пока системный вызов делает свою работу, пользовательский процесс обычно находится в ожидании. В таком случае системный вызов называют блокирующим.

Этот сценарий имеет несколько важных аспектов.

- Выполнением всей логики системных вызовов занимается одно ядро.
- Если системный вызов блокирующий, то вызывающая сторона должна дождаться завершения его работы. Неблокирующие системные вызовы очень быстро возвращаются, но пользовательскому процессу приходится выполнять дополнительные действия, чтобы проверить, доступны ли результаты.
- Аргументы вместе с входными и выходными данными копируются в пользовательское пространство и из него. Ввиду этого системные вызовы должны быть рассчитаны на прием крошечных значений и указателей в качестве входных аргументов.
- Ядро обладает полным привилегированным доступом ко всем ресурсам системы. Следовательно, должен существовать такой механизм, который проверяет, может ли пользовательский процесс выполнять тот или иной системный вызов.
- Похожее разделение существует между участками памяти, выделенными для пространств пользователя и ядра. Пользовательский процесс может обращаться только к памяти пространства пользователя. В ходе работы некоторых системных вызовов поток выполнения передается несколько раз.

Выполнение системного вызова напрямую, в обход стандартной библиотеки C

Пример, в котором системный вызов инициируется напрямую, в обход стандартной библиотеки C. Иными словами, программа выполняет системный вызов, не обращаясь к кольцу командной оболочки. Это считается плохим подходом, однако некоторые системные вызовы недоступны в libc и их пользовательское приложение может инициировать напрямую.

```
// Это нужно, чтобы использовать элементы не из состава POSIX
#define _GNU_SOURCE
#include <unistd.h>
// Это не является частью POSIX!
#include <sys/syscall.h>
int main(int argc, char **argv) {
    char message[20] = "Hello World!\n";
    // Инициирует системный вызов 'write', который записывает
    // некоторые байты в стандартный вывод.
    syscall(__NR_write, 1, message, 13);
    return 0;
}
```

```
$ gcc main.c -o a.out
$ ./a.out
Hello World!
```

Любая Unix-система предоставляет определенный метод для прямого выполнения системных вызовов. Например, в Linux для этого предусмотрена функция под названием syscall, размещенная в заголовочном файле <sys/syscall.h>.

В примере показана еще одна разновидность программы Hello World, в которой для передачи текста в стандартный вывод не используется libc. То есть не применяется функция printf, которая является частью кольца командной оболочки и стандарта POSIX. Вместо этого программа инициирует системный вызов напрямую, из-за чего данный код совместим только с Linux, но не с другими Unix-системами.

Выполнение системного вызова напрямую, в обход стандартной библиотеки C

```
$ strace ./a.out
execve("./a.out", ["./a.out"], 0x7ffc3cafcc60 /* 26 vars */) = 0
brk(NULL)                                   = 0x55f2800a1000
arch_prctl(0x3001 /* ARCH_??? */, 0x7fff203c6960) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd89ffbc000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
...
arch_prctl(ARCH_SET_FS, 0x7fd89fd87740) = 0
...
mprotect(0x7fd89ffa0000, 16384, PROT_READ) = 0
mprotect(0x55f27ee78000, 4096, PROT_READ) = 0
mprotect(0x7fd89fff6000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7fd89ffb3000, 34891)           = 0
write(1, "Hello World!\n", 13Hello World!)          = 13
exit_group(0)                           = ?
+++ exited with 0 +++
```

Здесь полужирным шрифтом выделено то место, где утилита strace записала системный вызов. Взгляните на возвращаемое значение (13). Оно означает, что системный вызов успешно записал 13 байт в заданный файл (в нашем случае это стандартный вывод).

Внутри функции syscall

```
#include <sysdep.h>
   /* Please consult the file sysdeps/unix/sysv/linux/x86-64/sysdep.h
for more information about the value -4095 used below.  */
   /* Usage: long syscall (syscall_number, arg1, arg2, arg3, arg4,
arg5, arg6)   We need to do some arg shifting, the syscall_number will
be in rax.  */
       .text
   ENTRY (syscall)
       movq %rdi, %rax      /* Syscall number -> rax.  */
       movq %rsi, %rdi      /* shift arg1 - arg5.  */
       movq %rdx, %rsi
       movq %rcx, %rdx
       movq %r8, %r10
       movq %r9, %r8
       movq 8(%rsp),%r9 /* arg6 is on the stack.  */
       syscall          /* Do the system call.  */
       cmpq $-4095, %rax    /* Check %rax for error.  */
       jae SYSCALL_ERROR_LABEL  /* Jump to error handler if error. */
       ret          /* Return to caller.  */

   PSEUDO_END (syscall)
```

Этот метод выполнения системного вызова более сложен, но сами инструкции выглядят лаконично и просто.

glibc не позволяет использовать определенные возможности ядер, которые поддерживают системные вызовы с более чем шестью аргументами и добавление такой поддержки потребует изменения данной библиотеки. К счастью, шести аргументов достаточно в большинстве случаев.

В листинге выше вслед за инструкцией movq ассемблерный код вызывает процедуру syscall. Он просто генерирует прерывание, которое активирует определенный участок ядра, предназначенный для его обработки.

Что касается функции syscall, то ее объявление написано на C, а определение — на ассемблере.

Добавление системного вызова в Linux

Опасность! Повторяйте только на виртуальных машинах (например, VirtualBox!)

Oracle VirtualBox Менеджер

Файл Машина Справка

Инструменты

Создать Добавить Настроить Сбросить Запустить

nuclei_change
Выключена

Общие

Имя: nuclei_change
ОС: Ubuntu (64-bit)

Система

Оперативная память: 4096 МБ
Процессоры: 4
Порядок загрузки: Жёсткий диск, Оптический диск, Гибкий диск
Ускорение: Nested Paging, Паравиртуализация KVM

Дисплей

Видеопамять: 128 МБ
Графический контроллер: VMSVGA
Сервер удалённого дисплея: Выключен
Запись: Выключена

Носители

Контроллер: IDE
Вторичное устройство IDE 0: [Оптический привод] Пусто
Контроллер: SATA
SATA порт 0: nuclei_change.vdi (Обычный, 29,96 ГБ)

Аудио

Аудиодрайвер: По умолчанию
Аудиоконтроллер: ICH AC97

Сеть

Адаптер 1: Intel PRO/1000 MT Desktop (NAT)

USB

USB-контроллер: OHCI, EHCI
Фильтры устройств: 0 (0 активно)

Общие папки

Отсутствуют

Описание

Включение ВМ ...
0%

nuclei_change

Разработка ядра

Разработка ядер имеет ряд отличий от того, как разрабатываются обычные программы на C. Шесть ключевых отличий в процессе разработки для ядра и для пользовательского пространства:

- Процесс ядра находится во главе системы и существует в единственном экземпляре. Это просто означает, что если ваш код приведет к сбою в ядре, то вам, скорее всего, придется перезагрузить компьютер и позволить ядру заново выполнить инициализацию.
- В кольце ядра нет никакой стандартной библиотеки C наподобие glibc! Иными словами, стандарты SUS и POSIX здесь не действуют. Например, занимаясь разработкой ядра в Linux, вы можете выполнять запись в буфер сообщений ядра с помощью функции printk. Но в FreeBSD для этого предусмотрено семейство функций printf, которые отличаются от своих аналогов из libc; они находятся в заголовочном файле <sys/system.h>.
- В ядре тоже можно читать и изменять файлы, но без использования функций из libc. У каждой Unix-системы есть собственный способ доступа к файлам из кольца ядра.
- В кольце ядра вы имеете полный доступ к физической памяти и многим другим возможностям.
- В ядре нет механизма системных вызовов. Системные вызовы — основной метод взаимодействия с кольцом ядра из пользовательского пространства. Поэтому в самом ядре в нем нет необходимости.
- Процесс ядра создается путем копирования образа ядра в физическую память. Этим занимается загрузчик. Вы не можете добавить новые системные вызовы, не прибегнув к созданию совершенно нового образа и инициализации его в ходе перезагрузки системы. Кое-какие ядра поддерживают модули, которые можно динамически добавлять и удалять, но с системными вызовами этого делать нельзя.

Все это показывает, что процесс разработки ядра отличается от обычной разработки на языке C. Тестирование написанной логики затруднено, и некачественный код может вывести из строя всю систему.

Написание демонстрационного системного вызова для Linux

Здесь мы напишем новый системный вызов для Linux. В Интернете есть много материала на эту тему, но мы возьмем за основу статью Adding a Hello World System Call to Linux Kerne (через VPN)!

Прежде всего мы должны загрузить самый свежий исходный код ядра Linux. Мы клонируем его из репозитория на GitHub и укажем нужный нам выпуск. **Linux** — это ядро. То есть оно может быть установлено только в кольце ядра Unix-подобной операционной системы. А вот дистрибутив Linux — другое дело. Он содержит определенные версии ядра Linux, библиотеки GNU libc и командной оболочки Bash (или GNU Shell). Дистрибутивы Linux обычно поставляются с набором пользовательских приложений во внешних кольцах, поэтому их можно считать полноценными операционными системами. Обратите внимание: дистрибутив, «дистр» и разновидность Linux — одно и то же.

```
$ sudo apt-get update
$ sudo apt-get install -y build-essential autoconf libncurses5-dev
libssl-dev bison flex libelf-dev git
...
...
```

apt — основной диспетчер пакетов в дистрибутивах Linux, основанных на Debian, а sudo — утилита, которая используется для запуска команд от имени администратора. Они доступны почти во всех Unix-подобных операционных системах.

Написание демонстрационного системного вызова для Linux

```
$ git clone https://github.com/torvalds/linux
$ cd linux
$ git checkout v5.3
```

Некоторые из этих каталогов могут показаться знакомыми: fs, mm, net, arch и т. д. Они могут существенно отличаться в зависимости от ядра; смысл в том, что большинство ядер имеют одну и ту же внутреннюю структуру.

Итак, получив исходники ядра, мы можем приступить к созданию нашего нового системного вызова. Но сначала нужно выбрать для него уникальный числовой идентификатор; в данном случае я назову его hello_world и назначу ему номер 999.

```
$ ls
total 760K
drwxrwxr-x 33 mikhail mikhail 4.0K Jan 28 2018 arch
drwxrwxr-x 3 mikhail mikhail 4.0K Oct 16 22:11 block
drwxrwxr-x 2 mikhail mikhail 4.0K Oct 16 22:11 certs
...
drwxrwxr-x 75 mikhail mikhail 4.0K Oct 16 22:11 fs
drwxrwxr-x 27 mikhail mikhail 4.0K Jan 28 2018 include
...
drwxrwxr-x 17 mikhail mikhail 4.0K Oct 16 22:11 kernel
drwxrwxr-x 13 mikhail mikhail 12K Oct 16 22:11 lib
-rw-rw-r-- 1 mikhail mikhail 429K Oct 16 22:11 MAINTAINERS
-rw-rw-r-- 1 mikhail mikhail 61K Oct 16 22:11 Makefile
drwxrwxr-x 3 mikhail mikhail 4.0K Oct 16 22:11 mm
drwxrwxr-x 69 mikhail mikhail 4.0K Jan 28 2018 net
-rw-rw-r-- 1 mikhail mikhail 722 Jan 28 2018 README
...
drwxrwxr-x 4 mikhail mikhail 4.0K Jan 28 2018 virt
drwxrwxr-x 5 mikhail mikhail 4.0K Oct 16 22:11 zfs
$
```

Написание демонстрационного системного вызова для Linux

Объявление нового системного вызова Hello World (include/linux/syscalls.h)

```
/*
 * syscalls.h - Linux syscall interfaces (non-arch
 *
 * Copyright (c) 2004 Randy Dunlap
 * Copyright (c) 2004 Open Source Development Labs
 * This file is released under the GPLv2.
 * See the file COPYING for more details.
 */
#ifndef _LINUX_SYSCALLS_H
#define _LINUX_SYSCALLS_H
struct epoll_event;
struct iattr;
struct inode;
...
asmlinkage long sys_statx(int dfd, const char __user *path, unsigned flags,
                          unsigned mask, struct statx __user *buffer);
asmlinkage long sys_hello_world(const char __user *str,
                                const size_t str_len,
                                char __user *buf,
                                size_t buf_len);
#endif
```

Согласно описанию это заголовочный файл с интерфейсами syscall, не зависящими от архитектуры. Это значит, Linux предоставляет один и тот же набор системных вызовов на всех архитектурах.

В конце файла мы объявили функцию нашего системного вызова, которая принимает четыре аргумента. Обратите внимание: входные аргументы обозначены как const, а выходные — нет. Кроме того, идентификатор __user означает, что указатели ссылаются на адреса в пользовательском пространстве. В нашем системном вызове 0 означает успех, а любое другое число — провал.

Написание демонстрационного системного вызова для Linux

```
#include <linux/kernel.h>   // Для printk
#include <linux/string.h>   // Для strcpy, strcat, strlen
#include <linux/slab.h>     // Для kmalloc, kfree
#include <linux/uaccess.h>  // Для copy_from_user, copy_to_user
#include <linux/syscalls.h> // Для SYSCALL_DEFINE4
SYSCALL_DEFINE4(hello_world, // Определение системного вызова
                const char __user *, str,     // Вводимое имя
                const unsigned int, str_len, // Длина вводимого имени
                char __user *, buf,           // Выходной буфер
                unsigned int, buf_len) {      // Длина выходного буфера
    // Переменная в стеке ядра должна хранить содержимое
    // входного буфера
    char name[64];
    // Переменная в стеке ядра должна хранить итоговое
    // выходное сообщение.
    char message[96];
    ...
    strcpy(message, "Hello "); // Формируем итоговое сообщение
    strcat(message, name);
    strcat(message, "!");
    // Проверяем, помещается ли итоговое сообщение в выходной двоичный буфер
    ...
    // Записываем отправленное сообщение в журнал ядра
    printk("Message: %s\n", message);
    return 0;
}
```

Теперь нам нужно определить наш системный вызов. Для этого необходимо сначала создать папку с именем hello_world в корневом каталоге.

Создание каталога hello_world

$ mkdir hello_world

$ cd hello_world $

Затем внутри hello_world нужно создать файл с именем sys_hello_world.c. Он должен иметь следующее содержимое

Написание демонстрационного системного вызова для Linux

Следующим шагом будет обновление еще одной таблицы. В архитектурах x86 и x64 предусмотрена лишь одна таблица для системных вызовов; мы должны добавить в нее наш новый системный вызов, чтобы сделать его доступным. Только после этого системный вызов можно будет использовать на компьютерах с архитектурами x86 и x64. Итак, мы должны добавить в таблицу системный вызов hello_word и имя его функции, sys_hello_world.

Откройте для этого файл arch/x86/entry/syscalls/syscall_64.tbl и добавьте в его конец следующую строчку.

Добавление нового системного вызова Hello World в таблицу системных вызовов

```
999   64   hello_world     __x64_sys_hello_world
```

После внесения этого изменения файл должен выглядеть так.

Системный вызов Hello World был добавлен в таблицу системных вызовов

```
$ cat arch/x86/entry/syscalls/syscall_64.tbl
...
...
546 x32 preadv2 __x32_compat_sys_preadv64v2
547 x32 pwritev2 __x32_compat_sys_pwritev64v2
999 64 hello_world __x64_sys_hello_world
```

Написание демонстрационного системного вызова для Linux

Для компиляции всех исходных файлов и создания итогового образа ядра Linux используется система сборки Make. Вы должны создать файл Makefile в каталоге hello_world. Этот файл должен содержать одну строчку следующего вида.

Файл Makefile для системного вызова Hello World

```
obj-y := sys_hello_world.o
```

Затем вам нужно добавить каталог hello_world в главный файл Makefile, который находится в корневом каталоге. Перейдите в корневой каталог ядра, откройте Makefile и найдите следующую строчку.

Строчка, которую необходимо изменить в корневом файле Makefile

```
core-y += kernel/certs/mm/fs/ipc/security/crypto/block/
```

Добавьте в данный список hello_world/. Это всего лишь перечень каталогов, которые должны собираться вместе с ядром.

Нам нужно добавить каталог системного вызова Hello World, чтобы он был включен в процесс сборки и стал частью итогового образа ядра. После редактирования эта строчка должна выглядеть следующим образом.

Строчка после редактирования

```
core-y += kernel/certs/mm/fs/hello_world/ipc/security/crypto/block/
```

Сборка ядра

Чтобы собрать ядро, нам нужно сначала вернуться в его корневой каталог и предоставить конфигурацию со списком возможностей и модулей, которые будут скомпилированы в процессе сборки.

Команда пытается сгенерировать нужную нам конфигурацию на основе включенной в текущее ядро Linux. Она берет имеющуюся конфигурацию и при наличии в нашем ядре, которое мы пытаемся собрать, новых значений просит вас их подтвердить. Для подтверждения достаточно нажать клавишу Enter.

Создание новой конфигурации на основе текущего активного ядра

```
$ make localmodconfig
...
...
#
# configuration written to .config
#
$
```

Сборка ядра

Теперь можно начать процесс сборки. Поскольку ядро Linux содержит много исходных файлов, сборка может занять несколько часов. Поэтому компиляцию нужно выполнять параллельно.

Если вы используете виртуальную машину, то, пожалуйста, сделайте ее процессор многоядерным. Это существенно ускорит процесс сборки.

```
$ make -j4
SYSHDR arch/x86/include/generated/asm/unistd_32_ia32.h
SYSTBL arch/x86/include/generated/asm/syscalls_32.h
HOSTCC scripts/basic/bin2c
SYSHDR arch/x86/include/generated/asm/unistd_64_x32.h
...
UPD include/generated/compile.h
CC init/main.o
CC hello_world/sys_hello_world.o
CC arch/x86/crypto/crc32c-intel_glue.o
...
LD [M] sound/pci/ac97/snd-ac97-codec.ko
LD [M] sound/pci/snd-intel8x0.ko
LD [M] sound/soundcore.ko
$
```

Сборка ядра

Не забудьте установить все необходимые пакеты, представленные в первой части этого раздела, иначе получите ошибки компиляции.

Как видите, начался процесс сборки, состоящий из четырех заданий, которые пытаются скомпилировать файлы C в параллель.

Создание и установка нового образа ядра

```
$ sudo make modules_install install
INSTALL arch/x86/crypto/aes-x86_64.ko
INSTALL arch/x86/crypto/aesni-intel.ko
INSTALL arch/x86/crypto/crc32-pclmul.ko
INSTALL arch/x86/crypto/crct10dif-pclmul.ko
...
run-parts: executing /et/knel/postinst.d/initam-tools 5.3.0+ /
boot/vmlinuz-5.3.0+
update-iniras: Generating /boot/initrd.img-5.3.0+
run-parts: executing /etc/keneostinst.d/unattende-urades 5.3.0+ /
boot/vmlinuz-5.3.0+
...
Found initrd image: /boot/initrd.img-4.15.0-36-generic
Found linux image: /boot/vmlinuz-4.15.0-29-generic
Found initrd image: /boot/initrd.img-4.15.0-29-generic done.
$
```

Сборка ядра

Новый образ ядра версии 5.3.0 был создан и установлен. Теперь мы можем перезагрузить компьютер. Но прежде не забудьте проверить версию текущего ядра, если не знаете ее.

Проверка версии текущего ядра

```
$ uname -r
4.15.0-36-generic
$
```

Теперь перезагрузим систему.

Перезагрузка системы

```
$ sudo reboot
```

В ходе загрузки системы будет выбрано и задействовано новое ядро. Стоит отметить, что загрузчик не подхватит старые ядра; следовательно, если у вас было ядро версии 5.3 и выше, то вам придется загрузить собранный вами образ вручную. Когда операционная система завершит загрузку, у вас должно быть новое ядро.

Проверка версии ядра после перезагрузки

```
$ uname -r
5.3.0+
$
```

Если все прошло как следует, то у вас должно быть загружено новое ядро.

Сборка ядра

Теперь мы можем вернуться к написанию программы на C, которая использует наш только что добавленный системный вызов Hello World.

```c
// Это нужно для использования элементов, не входящих в состав POSIX
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>// Это не является частью POSIX!
int main(int argc, char **argv) {
    char str[20] = "Kam";
    char message[64] = "";
    // Инициируем системный вызов hello world
    int ret_val = syscall(999, str, 4, message, 64);
    if (ret_val < 0) {
        printf("[ERR] Ret val: %d\n", ret_val);
        return 1;
    }
    printf("Message: %s\n", message);
    return 0;
}
```

```
$ gcc main.c -o main.out
$ ./main.out
Message: Hello Kam!
$
```

Как видите, мы инициировали системный вызов под номер 999. Мы подали на вход строку Kam и ожидаем получить в качестве приветствия Hello Kam!. Программа ожидает получения результата и выводит буфер с сообщением, записанный системным вызовом в пространстве ядра.

Сборка ядра

Использование команды dmesg для просмотра журнальных записей, сгенерированных системным вызовом Hello World

```
$ dmesg
...
[ 112.273783] System call fired!
[ 112.273786] Message: Hello Kam!
```

Мониторинг системных вызовов, инициированных примером

```
$ strace ./main.out
...
mprotect(0x557266020000, 4096, PROT_READ) = 0
mprotect(0x7f8dd6d2d000, 4096, PROT_READ) = 0
munmap(0x7f8dd6d26000, 27048) = 0
syscall_0x3e7(0x7fffe7d2af30, 0x4, 0x7fffe7d2af50, 0x40,
0x7f8dd6b01d80, 0x7fffe7d2b088) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 0), ...}) = 0
brk(NULL) = 0x5572674f2000
brk(0x557267513000)
...
exit_group(0) = ?
+++ exited with 0 +++
```

Сборка ядра

Передача длинного сообщения (больше 64 байт) системному вызову Hello World

```
int main(int argc, char** argv) {
    char name[84] = "A very very long message! It is really hard to
    produce a big string!";
    char message[64] = "";
    ...
    return 0;
}
```

Компиляция и запуск отредактированного примера

```
$ gcc main.c -o main.out
$ ./main.out
[ERR] Ret val: -1
```

Мониторинг системных вызовов

```
$ strace ./main.out
...
munmap(0x7f1a900a5000, 27048) = 0
syscall_0x3e7(0x7ffdf74e10f0, 0x54, 0x7ffdf74e1110, 0x40,
0x7f1a8fe80d80, 0x7ffdf74e1248) = -1 (errno 1)
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 0), ...}) = 0
brk(NULL) = 0x5646802e2000
...
exit_group(1) = ?
+++ exited with 1 +++
```

Ядра Unix

В этом разделе речь пойдет об архитектурах, которые были разработаны в ядрах Unix за последние 30 лет. Но прежде, чем обсуждать разные виды ядер, которых, к слову, не так уж и много, следует отметить, что процесс проектирования ядра никак не стандартизован.

Каждое ядро по-своему уникально. Но все они имеют общую черту: предоставляют доступ к своим возможностям через интерфейс системных вызовов. Однако каждое ядро имеет собственное видение того, как нужно обрабатывать эти вызовы.

Такое разнообразие и споры вокруг него стали одной из самых горячих тем в области компьютерных архитектур в 1990-х годах. В этих обсуждениях принимали участие большие группы людей. Самым знаменитым примером считаются дебаты между Таненбаумом и Торвальдсом.

Два основных направления в проектировании ядер Unix: монолит и микроядро. Существуют и другие архитектуры, такие как гибридные ядра, наноядра и экзоядра, и каждая из них имеет определенное назначение.

Монолитные ядра и микроядра

Монолитное ядро представляет собой один процесс с единым адресным пространством, содержащий более мелкие компоненты. Микроядра используют противоположный подход. ***Микроядро*** — минимальный процесс, из которого в пользовательское пространство вынесены такие компоненты, как файловая система, драйверы устройств и управление процессами; благодаря этому процесс ядра получается более компактным.

Обе архитектуры имеют свои преимущества и недостатки, благодаря чему стали темой одного из самых знаменитых обсуждений в истории операционных систем. Все началось еще в 1992 году, сразу после выпуска первой версии Linux. Начало дискуссии положил Эндрю С. Таненбаум публикацией в сети Usenet.

Публикация породила непримиримое противостояние между Таненбаумом, создателем Linux, Линусом Торвальдсом, и многими другими энтузиастами, которые позже стали одними из первых разработчиков Linux. Споры велись о природе монолитных ядер и микроядер. Они касались таких тем, как архитектура ядра и влияние на нее аппаратной платформы.

Монолитные ядра и микроядра

Ниже перечислены различия между монолитными ядрами и микроядрами.

- Монолитное ядро представляет собой единый процесс, содержащий все возможности, предоставляемые этим ядром. У микроядер все иначе: каждую возможность они предоставляют в виде отдельного процесса.
- Процесс монолитного ядра находится в пространстве ядра, тогда как в микроядре серверные процессы обычно вынесены в пространство пользователя. Серверными называют процессы, которые предоставляют доступ к компонентам ядра, таким как диспетчер памяти, файловая система и т. д.
- Монолитные ядра, как правило, быстрее. Это объясняется тем, что все их функции выполняются внутри одного процесса. Микроядрам же нужно организовывать обмен сообщениями между пространствами пользователя и ядра, что приводит к большему количеству системных вызовов и переключений контекста.
- В монолитной архитектуре все драйверы устройств загружаются прямо в ядро. С микроядрами все иначе, поскольку все драйверы устройств и многие другие компоненты выполняются в пространстве пользователя.
- Для взлома монолитного ядра и, следовательно, всей системы достаточно внедрить в него небольшой фрагмент вредоносного кода.
- В монолитной архитектуре малейшее изменение исходного кода требует перекомпиляции всего ядра и создания его нового образа.

Один из самых известных примеров микроядра — MINIX. Этот проект, написанный Эндрю С. Таненбаумом, изначально создавался в образовательных целях.

Linux

Вы уже познакомились с ядром Linux, мы разрабатывали новый системный вызов. Здесь же мы сделаем акцент на монолитности Linux и на том факте, что в процессе ядра находятся все его возможности.

Однако должен существовать способ добавить в ядро новую функциональность, не прибегая к его перекомпиляции. Этого нельзя достичь за счет новых системных вызовов, поскольку, как вы уже видели, для их добавления нужно отредактировать много важных файлов, а это означает компиляцию ядра заново.

Но есть другой подход. Он предусматривает написание и динамическое подключение к ядру модулей. Именно о модулях пойдет речь в следующем подразделе, а затем мы попробуем написать собственный модуль для Linux.

Модули ядра

Монолитные ядра обычно предусматривают механизм, который позволяет динамически подключать к активному ядру новые компоненты. Эти подключаемые компоненты называются модулями ядра.

В отличие от серверных процессов в микроядрах, которые фактически являются отдельными процессами, взаимодействующими между собой по IPC, модули представляют собой объектные файлы ядра, скомпилированные и готовые к загрузке в его процесс. Эти файлы можно собрать статически и сделать их частью образа ядра; но вы также можете загружать их динамически, когда ядро уже работает.

Обратите внимание: по своей сути объектные файлы ядра аналогичны объектным файлам, которые создаются в процессе разработки на языке C.

В целом общение с модулями ядра в Linux и некоторых похожих операционных системах происходит тремя способами.

- Файлы устройств в каталоге /dev. Модули ядра в основном разрабатываются для использования в качестве драйверов устройств. Вот почему самый распространенный способ взаимодействия с ними — использование файлов внутри каталога /dev.
- Содержимое procfs. Файлы в каталоге /proc можно использовать для чтения метаинформации об отдельных модулях ядра. Эти файлы также позволяют отправлять модулям метаинформацию или управляющие команды.
- Содержимое sysfs. Это еще одна файловая система в Linux, которая позволяет управлять пользовательскими процессами и другими компонентами, относящимися к ядру (включая модули ядра), с помощью скриптов или вручную. Ее можно считать новой версией procfs.

Создание нового модуля ядра для Linux

Здесь мы напишем новый модуль ядра для Linux под названием Hello World. Он будет создавать запись в procfs. Затем мы используем эту запись, чтобы прочитать строку с приветствием.

Вы научитесь писать, компилировать и загружать модули в ядро, выгружать их оттуда и читать данные из записи в procfs. ***Основное назначение этого примера*** — дать вам возможность набраться опыта и стать более самостоятельным разработчиком.

Модули ядра компилируются в объектные файлы, которые можно загружать непосредственно в активное ядро. Для этого не нужно перезагружать систему. Главное, чтобы ваш модуль не натворил беды в ядре и не вывел его из строя. То же самое можно сказать о выгрузке модуля.

Первым делом нужно создать каталог, в котором будут находиться все файлы, относящиеся к модулю ядра.

```
$ mkdir example
$ cd example
$
```

Модуль ядра Hello World (example/hwkm.c) (24_3)

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
// Структура, ссылающаяся на файл proc
// Функция обратного вызова для чтения файла
// Обратный вызов для инициализации модуля
// Обратный вызов для выхода из модуля
// Определение обратных вызовов для работы с модулем
```

Создание нового модуля ядра для Linux

Обратите внимание: на данном этапе код нашего модуля имеет неограниченный доступ к почти любым компонентам ядра и из него в пользовательское пространство может утечь какая угодно информация. Это серьезная дыра в безопасности, вследствие чего вам следует поближе познакомиться с рекомендациями по написанию безопасных модулей ядра.

Чтобы скомпилировать приведенный выше код, нам нужен подходящий компилятор, с помощью которого мы потенциально можем выполнить компоновку с нужными нам библиотеками. Чтобы упростить себе жизнь, создадим файл с именем Makefile, который вызовет все необходимые инструменты для сборки нашего модуля ядра. ***Содержимое файла Makefile для модуля ядра Hello World***

```
obj-m += hwkm.o
all:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Создание нового модуля ядра для Linux

Сборка модуля ядра Hello World

```
$ make
make -C /lib/modules/54.318.0+/build M=/home/m/example
modules
make[1]: Entering directory '/home/m/linux'
CC [M] /home/m/example/hwkm.o
Building modules, stage 2.
MODPOST 1 modules
WARNING: modpost: missing MODULE_LICENSE() in /home/m/example/hwkm.o
see include/linux/module.h for more information
CC /home/m/hwkm.mod.o
LD [M] /home/m/hwkm.ko
make[1]: Leaving directory '/home/m/linux'
```

Как видите, в результате компиляции кода получается объектный файл, который затем компонуется с другими библиотеками в файл .ko. Среди сгенерированных результатов должен находиться файл с именем hwkm.ko.

Обратите внимание на расширение .ko. Оно означает, что выходной файл является объектным файлом ядра. Это нечто вроде разделяемой библиотеки, которую можно динамически загружать в ядро, где она будет выполняться.

Создание нового модуля ядра для Linux

Список существующих файлов после сборки модуля ядра Hello World

```
$ ls -l
total 556
-rw-rw-r-- 1 m m 154 Oct 19 00:36 Makefile
-rw-rw-r-- 1 m m 0 Oct 19 08:15 Module.symvers
-rw-rw-r-- 1 m m 1104 Oct 19 08:05 hwkm.c
-rw-rw-r-- 1 m m 272280 Oct 19 08:15 hwkm.ko
-rw-rw-r-- 1 m m 596 Oct 19 08:15 hwkm.mod.c
-rw-rw-r-- 1 m m 104488 Oct 19 08:15 hwkm.mod.o
-rw-rw-r-- 1 m m 169272 Oct 19 08:15 hwkm.o
-rw-rw-r-- 1 m m 54 Oct 19 08:15 modules.order
$
```

Мы использовали инструменты сборки модулей из ядра Linux версии 5.3.0. Вы можете получить ошибку, если попытаетесь скомпилировать этот пример с помощью ядра, версия которого ниже 3.10.

Загрузка и установка модуля ядра Hello World

```
$ sudo insmod hwkm.ko
$
```

Создание нового модуля ядра для Linux

Проверка журнальных сообщений ядра после установки модуля

```
$ dmesg
...
...
[ 7411.519575] Hello World module is loaded.
$
```

Итак, модуль загружен, и уже должен быть создан файл /proc/hwkm. Мы можем прочитать его с помощью команды cat. ***Чтение файла /proc/hwkm с использованием команды cat***

```
$ cat /proc/hwkm
Hello World From Kernel Module!
$ cat /proc/hwkm
Hello World From Kernel Module!
```

Выгрузка модуля ядра Hello World

```
$ sudo rmmod hwkm
```

Проверка журнальных сообщений ядра после выгрузки модуля

```
$ dmesg
...
...
[ 7411.519575] Hello World module is loaded.
[ 7648.950639] Goodbye World!
```

Создание нового модуля ядра для Linux

Особенности модулей ядра, с которыми мы успели познакомиться:

- модули ядра можно загружать и выгружать, не прибегая к перезагрузке компьютера;
- после загрузки модули становятся частью ядра и получают доступ ко всем его компонентам и структурам. Это можно считать уязвимостью, но ядро Linux имеет средства защиты от загрузки нежелательных модулей;
- модули ядра можно компилировать отдельно. А вот системные вызовы требуют перекомпиляции всего ядра, на что может легко уйти несколько часов.

Наконец, модули ядра могут пригодиться при разработке кода, который необходимо выполнять в самом ядре и инициировать с помощью системных вызовов. Сначала вы можете реализовать свою логику в виде модуля, а затем, протестировав ее должным образом, сделать доступной через настоящий системный вызов.

Процесс разработки системных вызовов с нуля может быть довольно трудоемким, поскольку требует регулярной перезагрузки компьютера. Но если сначала написать и протестировать логику в виде модуля, то это упростит разработку ядра. ***Стоит отметить:*** если ваш код нестабильный, то неважно, где он находится — в модуле или в системном вызове; он может привести к сбою в ядре, что потребует перезагрузки системы.

Адженда

История и архитектура Unix	Системные вызовы и ядра	Поиск подстрок (продолжение)
30 минут	30 минут	30 минут

Суффиксные массивы

Мотивация:

T – длина текста, P – длина подстроки

Скорость поиска подстроки O(P)

Для суффиксных деревьев скорость V ~ C·P, однако константа C – велика.

Хочется уменьшить константы, для этого есть решение – суффиксные массивы.

T = abacaba	**0**
bacaba	**1**
acaba	**2**
caba	**3**
aba	**4**
ba	**5**
a	**6**

В лексикографическом порядке (6, 4, 0, 2, 5, 1, 3) = **SA (Suffix array)**

Как найти например префикс P = ab быстро.

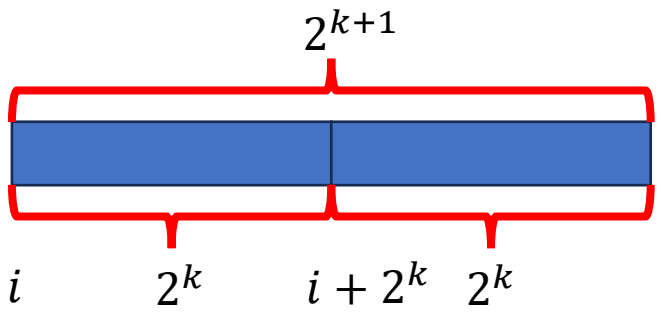
В суффиксном массиве можно использовать оптимизированный бинарный поиск, найти первое вхождение и все остальные вхождения будут подряд. [4, 0, 2]. Пока скорость O(P log T).

Построение суффиксного массива за T log T.

Идеи:

1. T = abacaba____..., представим, что в конце бесконечное количество пробелов, чтобы сравнивать строки равной длины. $\alpha_i = T[i, i + T]$.
2. Будем строить алгоритм из фаз: 0, 1, 2. Для фазы i цель отсортировать строки, длиной $T[k, k + 2^k]$, очевидно размер фаз O(logT).
3. Метка (label) – $W \rightarrow \{0, \dots, l - 1\}, label(\alpha) < label(\beta) \Leftrightarrow \alpha < \beta$

Суффиксные массивы



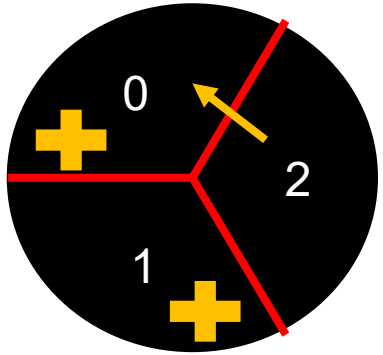
Фаза 0: $\forall c \in T\ label(c) = atoi(c)$
Фаза k → фаза k + 1:
Нужно сделать cmp (компаратор)
i → T[i, $i+2^{k+1}$] (T[i: $i+2^k$], T[$i+2^k$, $i+2^k$])
Итого: $label^k(i), label^k(i+2^k), i)$ – radix sort(O(T))
Очевидно, что num(label) ~ T, так как зачем больше. Главное подобрать такие метки.

Усложним, нам нужно построить за **O(T) сделать все и сразу**.
SA: {0, …, n – 1} → {0, …, n – 1}, то есть это просто некая перестановка чисел.
Суффиксный массив – это массив, элементами которого являются позиции в тексте. То есть отображение ранга в позицию. $SA^{-1}: position \rightarrow rank$
Пример: mississippi
mississippi _ _ _ - дополнили 3 пробелами. Пробел – самый маленький символ.
Разобьем всё на триплеты. И новый алфавит из трех символов. $\Sigma' = \{T[i: i+3\}$

$$\text{примеры нового алфавита} \begin{cases} 0 \rightarrow ___ \\ 1 \rightarrow i__ \end{cases}$$

T′ = mississippi _ _ _ _ ississippi _ _ = $\alpha\$\beta$ ~ 2/3n + O(1)
SA′ → SA хотим из бесполезной строки строить полезную, супернепонятно, нелогично
На самом деле это нужно для ускорения компаратора для сравнения подстрок.

Суффиксные массивы



Рекурсия:

Существуют 3 типа суффиксов, так как делим на тройки.

Если 2 суффикса типа 0 или 1 → сравнить в T′ до пробелов или после.

Если 2 суффикса типа 2 → сравнить по символу и перейти к сравнению типа 0.

Сортируем все radix sort

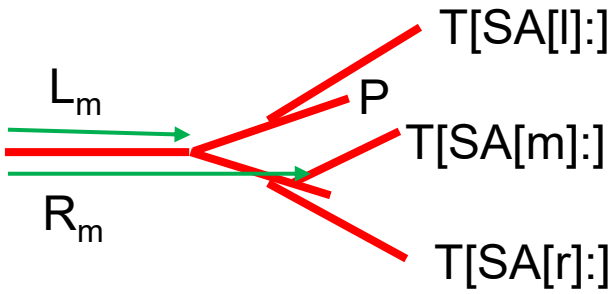
Далее $SA' \rightarrow SA_{01} \cup SA_2$ - слияние двух отсортированных списков – линейное время

Время T(n) = T(2/3n) + O(n) = O(n)

Подробно об алгоритме

Как улучшить скорость алгоритма от O(P log T) до O(P + log T).

Есть бинарный поиск на интервале [l, r], m ∈ [l, r)



l, r – ranks

L_p = |lcp(P, T[SA[l]:])|

R_p = |lcp(P, T[SA[r]:])|

L_m = |lcp(T[SA[m]:], T[SA[l]:])|

R_m = |lcp(T[SA[m:], T[SA[r]:])|

lcp – наибольший общий префикс

Хотим знать все эти параметры.

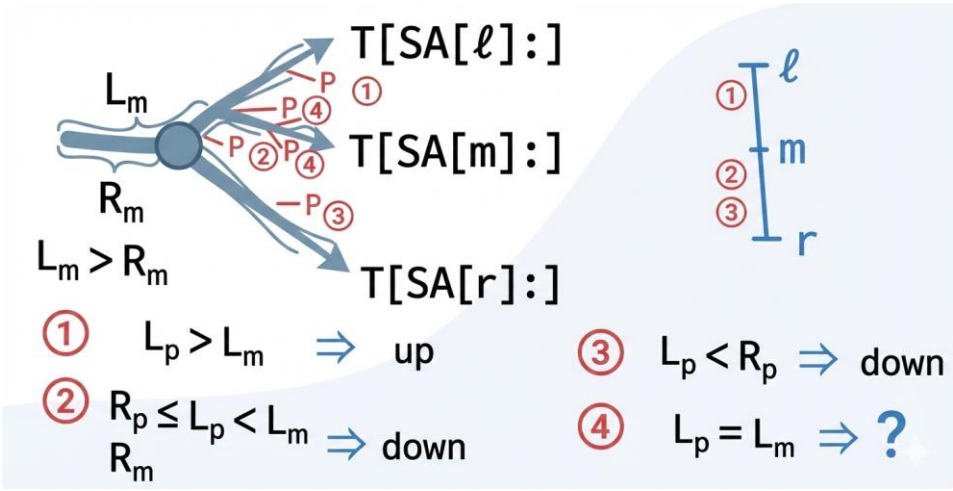
L_m может быть больше меньше или равен R_m

P относительно T[SA[m]:] может быть где угодно

Можно рассмотреть все случаи и понять куда пойдет бинарный поиск.

Суффиксные массивы

1 из случаев



4 случай – просто сравнить P и T[SA[m]:]
Очевидно что это не больше O(P).
Как посчитать R_m и L_m.
Очевидно, что поиск это в любом случае функция от двух аргументов f(l, m(l, r), r) = f(l, r).
Бинарный поиск проходит по каждому элементу по разу, значит если просто хранить препровождении некие R_m и L_m, то память O(n).
Уже не так грустно.

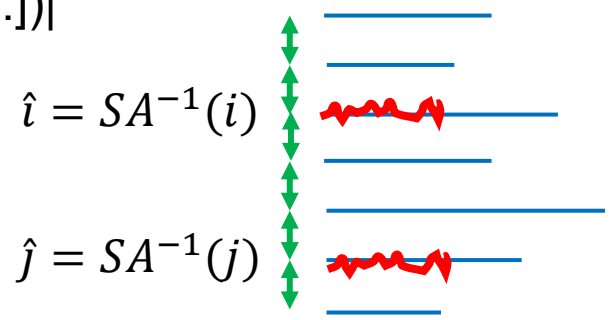
КАК??? Пойдём дальше – нужна некая структура, такая, чтобы после обработки текста она могла за O(1) выдавать lcp и при этом быть O(n) по памяти.

LCP(i,j) = |lcp(T[i:],T[j:])|

pos

$\hat{\imath}, \hat{\jmath} - rank$

RMQ



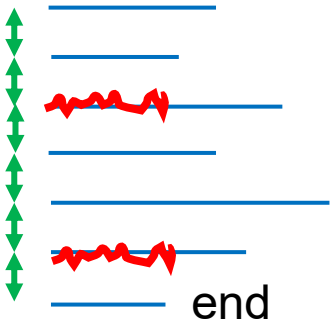
Для соседних пар можно посчитать lcp, то задача бы свелась к задаче RMQ.
Очевидно, что LCP(i, j) ≥ min_prefix
Но и LCP(i, j) < min_prefix + 1, так как prefix идет по возрастанию.
Самая большая сложность все ***lcp за O(n)?***

Суффиксные массивы

Ответ на вопрос в статье. Как посчитать все ***lcp за O(n)?***

[Kasai T. et al. Linear-time longest-common-prefix computation in suffix arrays and its applications //Annual Symposium on Combinatorial Pattern Matching. – Berlin, Heidelberg : Springer Berlin Heidelberg, 2001. – С. 181-192.]

LCP массив что это? $LCP[i] = |lcp(T[SA[i]:], lcp(T[SA[i+1]:])$



$rank \xrightarrow{SA}$ pos pos $\xrightarrow{SA}$ $rank \xrightarrow{SA^{-1}}$ pos

$LCP'^{[0]}, LCP'[1], \ldots.$

Давайте поменяем смысл переменной. $LCP'[i] = |lcp(T[i:], lcp(T[SA[SA^{-1}[i]+1]:])$

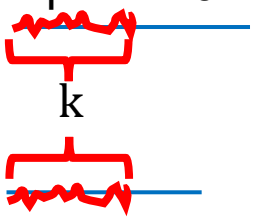
Далее очевидно, что $LCP[i] = LCP'[SA[i]]$

Проблема только в LCP'[end]

Лемма 22.1. $LCP'[i+1] \geq LCP'[i] - 1.$

Доказательство. Пусть $\mathrm{k} = LCP'[i]$.

При k = 0 и 1 – очевидно.

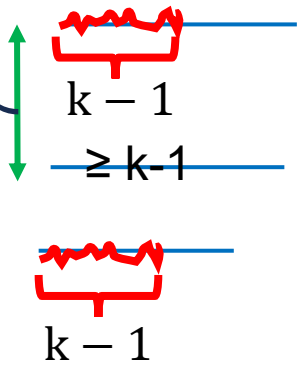


$T[i:]$

Убрали первый символ

$T[SA[SA^{-1}[i]+1]:]$

То что надо – следующий элемент



$T[i+1:]$

$T[SA[SA^{-1}[i+1]+1]:]$

$T[SA[SA^{-1}[i]+1]+1:]$

Очевидно лексикографический порядок

Помним что там между i и j RMW, очевидно между ними ≥ k-1

Вывод, чтобы знать о строке все, нужно знать SA + LCP + RMQ

Преобразование Барроуза-Уиллера

T = abacaba$, $\forall x \in \text{alphabet}\ \$ < x$

BWT(T) = T’

$T = \alpha\beta \rightarrow \beta\alpha$

Что оно делает? Циклические перестановки в лексикографическом порядке

Пример:

$$\begin{pmatrix} \text{abacaba\$} \\ \text{bacaba\$a} \\ \text{acaba\$ab} \\ \text{caba\$aba} \\ \text{aba\$abac} \\ \text{ba\$abaca} \\ \text{a\$abacab} \\ \text{\$abacaba} \end{pmatrix} \xrightarrow{\text{Лексикографический порядок}} \begin{pmatrix} \text{\$abacaba} \\ \text{a\$abacab} \\ \text{aba\$abac} \\ \text{acaba\$ab} \\ \text{abacaba\$} \\ \text{ba\$abaca} \\ \text{bacaba\$a} \\ \text{caba\$aba} \end{pmatrix}$$

T’ = abcb$aaa - образ Барроуза-Уиллера

Ключевые свойства BWT:

- Концентрация повторяющихся символов;
- Улучшение сжатия;
- Обратимость;
- Помогает в работе поиска подстроки (это тоже индекс)
- Часть алгоритма bzip2
- BWT + RLE – дает сжатие. RLE Пример: aaabb →a3b2

Очевидно, что восстановить обратно не просто..

abcb$aaa – последний столбец. Очевидно первый столбец – по возврастанию $aaaabbc

Так как перестановка циклическая – знаем все биграммы

abcb$aaa + $aaaabbc = a$ ba ca ba $a ab ab ac. Теперь зная что все по алфавиту можем восстановить второй столбец, отсортировав биграммы. Далее по аналогии триграммы и т. д. Очевидно наша строка – заканчивающаяся на доллар.

Мы хотим алгоритм за O(n) все

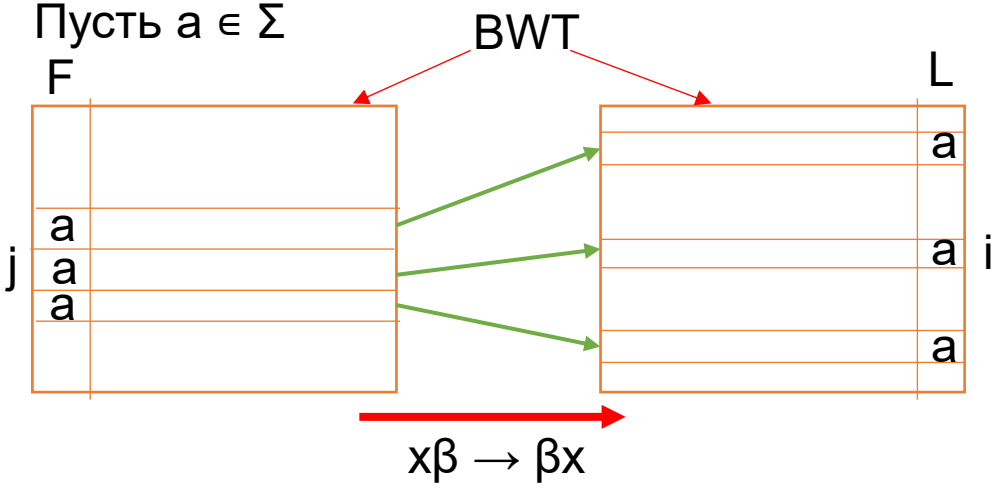
Преобразование Барроуза-Уиллера

O(n) T → T’

Идея – удвоение строчки TT + суффиксный массив + выкинуть половину суффиксов.

O(n) T’ → T

Пусть a ∈ Σ



Пусть L – last, F – first, тогда

LF(i) = j
FL(j) = I

} Биекции (без доказательства)

Как вычислить LF и FL за O(1)?

a := F[j]	Найти a
k := j – Start[a]	Понять какое оно по счету
i := $Select_L(a, k)$	Выбор a по счету в L

a := L[i]	Найти a
k := $Rank_L(a, i)$	Сколько a на префиксе i
j := Start[a] + k	Находим нужный a

i	a	bc	$
$FL^2(i)$	c	$a	b
$FL(i)$	b	c $	a

$i \to FL(i) \to FL^2(i) \to \cdots$

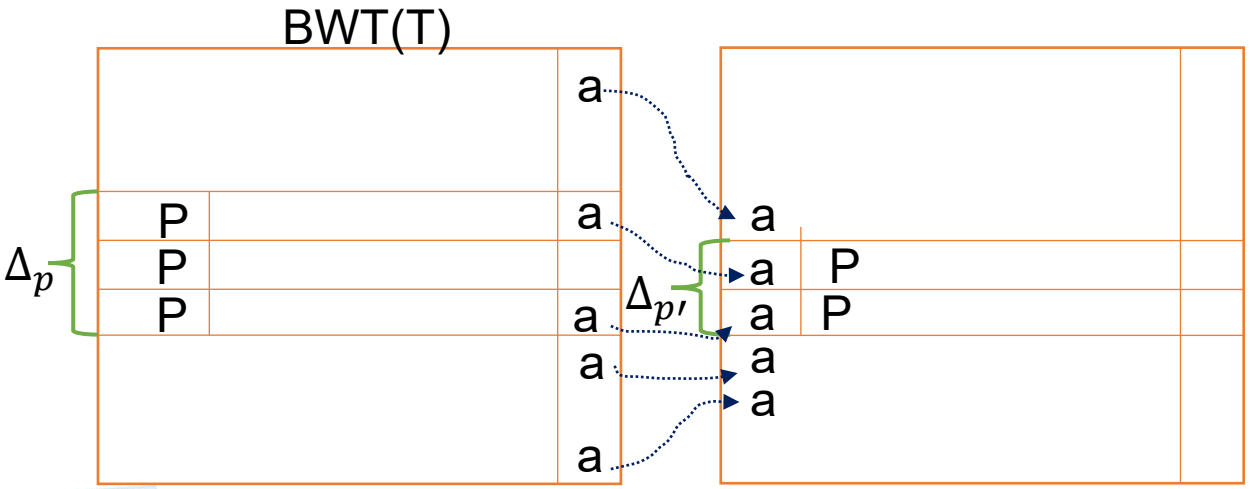
Есть структуры данных для Rank-Select.
Как просто посчитать Select?
Вектор векторов – символ 1 вектор, индекс-позиция – внутренний вектор. И проход по L вначале.

Преобразование Барроуза-Уиллера

BWT как помогает в работе поиска подстроки? (почему это тоже индекс?)

Пусть есть текст T = abac$, T’ = BWT(T)

P – образец, который ищется в строке T



P будут лежать подряд.

Для каждой строки можем запомнить циклический сдвиг

Далее все просто, но это долго.

Теперь нам надо узнать все за **O(P)**.

Если P = a

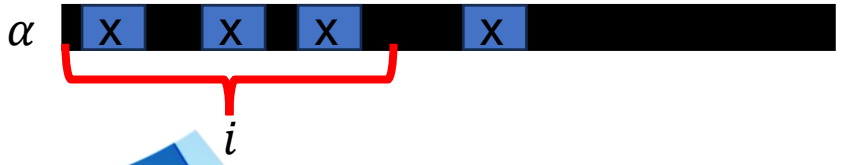
У нас есть массив Start начал символов:

[Start[a], Start[a+1]]

Теперь – знаем P, считаем P’ = aP. Хотим $\Delta_p \xrightarrow{O(1)} \Delta_{p'}$

В нашем примере слева $|\Delta_{p'}| = 2$

Находим с помощью Start[a] и $Rank_L(i, a)$



$Rank(i, x) = \#\{j < i | \alpha_i = x\}$

Если препроцессинг O(n|Σ|) слов - легко, хотим быстрее и экономнее по памяти.

|Σ| = 2 заполним битами и будем искать единицы

Итог: O(n) слов – много памяти. Хотим o(n) битов.

Преобразование Барроуза-Уиллера

RAM машина – очевидно адресное пространство должно быть не меньше логарифма длины всей памяти

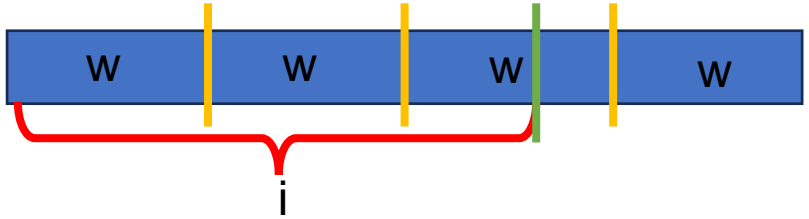
w ≥ log n

Сколько единиц в машинном слове w? Посчитать за O(1) – popcount.

Руками:

- Разбить пополам и предпросчитать всего памяти $2^{\frac{w}{2}} \sim \sqrt{n}$

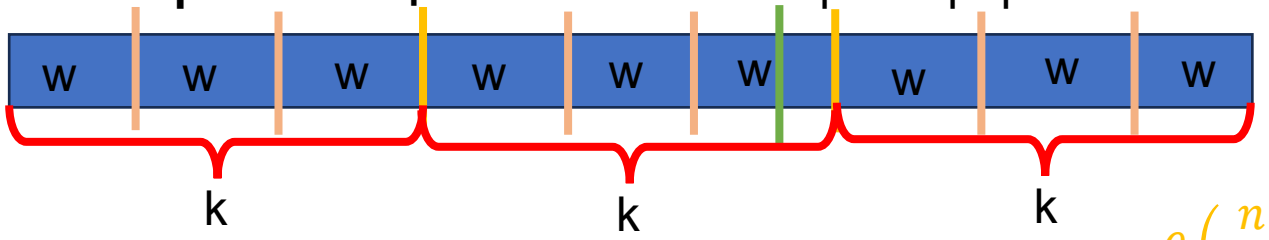
Теперь хотим предпросчитать на префиксе – битовая маска или сдвиги – O(1).

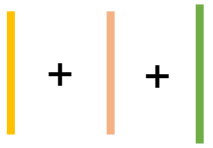


w ≥ log n

$$o(n) + \theta\left(\underbrace{\frac{n}{w}}_{\text{Количество ячеек}} \cdot \underbrace{\log n}_{\text{Размер ячейки}}\right) = \theta(n)\ bits$$

Как потратить еще меньше бит? Проитерировать





$$\theta\left(\frac{n}{wk} \cdot \log n\right) + \theta\left(\frac{n}{w} \cdot \log wk\right) + o(n) = |k = \log n|$$

$$= \theta\left(\frac{n}{w}\right) + \theta\left(\frac{n}{w} \cdot (\log w + \log n)\right) + o(n) = o(n)$$

Преобразование Барроуза-Уиллера

А теперь |Σ| > 2

Используем **«Wavelet Tree»**

0001111001111

Пусть Σ = abcdef, T = aabcdcdabccef in abcdef

00000011

aabab in ab
00101

cdcdccef in cdef

cdcdcc in cd
010100

ef in ef
01

Цепочка задач rank wavelet tree

$$\sum_i depth_i = \sum_{x \epsilon \Sigma} n(x) \cdot depth(x) \leq n(H+1)$$

Сколько занимает времени?
Ответ: O(log|Σ|)
Сколько занимает памяти?
rank структура
Ответ: O(n log|Σ|) bits
Можно алфавит делить произвольно и получить лучше space на может чуть хуже time.
По факту нам для каждого символа глубина нужна меньше чем больше частота $\sum_i (depth)_i$ - дерево Хаффмана скелет **Wavelet Tree**

Wavelet Tree занимает столько же памяти, сколько и сжатая Хаффманом строчка.
Можно из битов восстановить обратно строку?
Очевидно ***можно (простое дз – как?)***, то есть оно еще и сжимает строку.